S2D-09: Naming Standards & Organization

> **Series:** S2D | **Notebook:** 9 of 9 | **Created:** January 2026 | **Last Updated:** 01/30/2026

Overview

Establishing and following a consistent naming standard is essential to a successful monitoring deployment. This notebook provides recommendations for naming assets created during the Splunk to Dynatrace migration.

Table of Contents

Prerequisites

Requirement	Details
Knowledge	Understanding of assets to be migrated
Coordination	Agreement with application teams on naming

Learning Objectives

By the end of this notebook, you will be able to:

1. Apply consistent naming standards to migrated assets
2. Structure lookup tables for proper access control
3. Organize dashboards and alerts by application
4. Use event name templates effectively

Dashboard Naming

Standard Format

```
[app_name] Dashboard Title
```

The `app_name` should match your organization's application naming convention (e.g., wave-assigned app name).

Examples

Correct	Incorrect
`[EasyTravel] Business Overview Dashboard`	`Business Overview Dashboard`
`[PaymentService] Transaction Monitoring`	`Payment Transaction Dashboard`
`[OrderManagement] Error Analysis`	`Order Errors`

Alert Naming

Davis Anomaly Detectors have two name fields:

1. Configuration Name

The name of the anomaly detector configuration.

Format:

```
[app_name] alert name from splunk
```

Example:

```
[EasyTravel] JourneyService High Error Log Count
```

2. Event Name (Triggered Alert Title)

The name shown when the alert fires. Can include placeholders for dynamic content.

Format:

```
[app_name] [optional dimensions] - alert name from splunk
```

Available Placeholders:
- `{host.name}` - Affected host
- `{dt.entity.service}` - Service entity
- `{k8s.deployment.name}` - Kubernetes deployment

Examples:

Event Name Template	Result When Triggered
`[EasyTravel] P2 - {host.name} - High Error Count`	`[EasyTravel] P2 - app-server-01 - High Error Count`
`[Payment] {k8s.deployment.name} - Slow Response`	`[Payment] checkout-service - Slow Response`

Report Naming

Workflows, notebooks, or dashboards created to reproduce Splunk report functionality should be clearly identified.

Standard Format

```
[Report] [app_name] Report Title
```

Examples

Correct	Incorrect
`[Report] [EasyTravel] Daily Transaction Report`	`EasyTravel Daily Report`
`[Report] [Inventory] Weekly Stock Summary`	`Weekly Stock Summary`

The `[Report]` prefix differentiates these from:
- Migrated dashboards (for operational monitoring)
- Workflows created for alerting purposes

Lookup Table Naming

Lookup tables should be organized by application to enable access control.

File Path Format

```
/lookups/[app_name]/[table_name]
```

Structure

```
/lookups/
├── easytravel/
│   ├── users
│   ├── locations
│   └── error_codes
├── payment/
│   ├── transaction_types
│   └── currency_codes
└── inventory/
    ├── warehouses
    └── product_categories
```

Benefits

- Access control can be applied at the application directory level
- Teams only have access to their own lookup tables
- Clear ownership and organization

Metric Naming

Log-based metrics created via OpenPipeline should follow a consistent pattern.

Recommended Format

```
log.[app_name].[metric_description]
```

Examples

Metric Name	Description
`log.easytravel.error_count`	Error log count
`log.payment.transaction_duration`	Transaction duration
`log.inventory.stock_updates`	Stock update events

Best Practices

- Use lowercase with underscores
- Be descriptive but concise
- Include the source type (`log.`)
- Group by application

Workflow Naming

Alerting Workflows

```
[Alert] [app_name] Alert Description
```

Example: `[Alert] [EasyTravel] High Error Rate Check`

Automation Workflows

```
[Auto] [app_name] Automation Description
```

Example: `[Auto] [Inventory] Daily Report Generation`

Remediation Workflows

```
[Remediate] [app_name] Remediation Description
```

Example: `[Remediate] [Web] Pod Restart on OOM`

Summary Table

Asset Type	Format	Example
Dashboard	`[app_name] Title`	`[EasyTravel] Business Overview`
Alert Config	`[app_name] alert_name`	`[EasyTravel] High Error Count`
Event Name	`[app_name] [dims] - alert`	`[EasyTravel] P2 - {host} - Errors`
Report	`[Report] [app_name] title`	`[Report] [EasyTravel] Daily Stats`
Lookup	`/lookups/[app]/[table]`	`/lookups/easytravel/users`
Metric	`log.[app].[metric]`	`log.easytravel.error_count`
Workflow (Alert)	`[Alert] [app] desc`	`[Alert] [Payment] Slow Response`
Workflow (Auto)	`[Auto] [app] desc`	`[Auto] [Inventory] Report Gen`

Implementation Recommendations

1. Document Your Standards

Create a reference document that all teams can access.

2. Enforce During Migration

Review naming during the migration process, not after.

3. Use Templates

Provide templates for common asset types to ensure consistency.

4. Regular Audits

Periodically review assets for naming compliance.

5. Automate Where Possible

Use infrastructure-as-code (Monaco, Terraform) to enforce naming through configuration.

Series Summary

This completes the S2D (Splunk to Dynatrace) migration series. You should now be able to:

1. ✅ Validate log data availability (S2D-02)
2. ✅ Translate SPL queries to DQL (S2D-03)
3. ✅ Migrate alerts to Davis Anomaly Detectors (S2D-04)
4. ✅ Use workflows for specialized alerting (S2D-05)
5. ✅ Handle extended timeframes with ArrayMovingSum (S2D-06)
6. ✅ Request metric extraction for performance (S2D-07)
7. ✅ Convert dashboards effectively (S2D-08)
8. ✅ Apply consistent naming standards (S2D-09)

References

- Dynatrace Documentation
- DQL Reference
- Davis Anomaly Detectors
- Workflows

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.